E7: Introduction to Computer Programming
Midterm 1A

Monday, October 02

Full Name

Student ID Number

Instructions:

- You have 1 hour and 20 minutes to complete this exam.
- The exam is closed book, closed notes, closed computer, closed calculator.
- You may only use one 8.5" × 11" sheet of notes (both sides). No other resources are permitted.
- Mark your answers on the exam paper within the designated spaces. Responses written on scratch paper or outside of the specified answer spaces will not be graded.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- The number of points associated with each part of each question is indicated.
- The exam is worth a total of 100 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only chose one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.

Good luck!

"We're rooting for you. You've got this!"

Question 1: Let's Begin with a Warm-up (30 points)

Each part below is independent of the other.

(a) (2 pt) What programming language(s) will be introduced in E7 this semester? For questions with circular bubbles, you should select exactly *one* choice.

◯ Python
◯ MATLAB
◯ All of the above

(b) (2 pt) What programming language(s) will be introduced in E7 this semester? For questions with square boxes, you may select *multiple* choices.

☐ Python
☐ MATLAB
☐ LOLCODE

(c) (4 pt) Select the expression(s) that can be used in blank (c) to calculate: $a = \frac{e^2}{3p} + e$, where $e = 2.7182...$ is the base of natural logarithms.

```
import math
p = 4
a = ______(c)______
```

☐ `math.exp(2) / (3p) + math.exp(1)`
☐ `math.exp(2) / 3 / p + math.exp(1)`
☐ `math.exp(2) / (3 * p) + e`
☐ `math.exp(2) / 3 * p + math.exp(1)`
☐ `math.exp(2) / (3 * p) + math.exp(1)`
☐ `e ** 2 / (3 * p) + e`

(d) (3 pt) Redefine the following function using a single line of code. Your line of code should replace the function definition AND its body, all in a single line of code.

```
def fxn(n):
    if n > 3:
        return True
    else:
        return False
```

For each expression below, write the output that will be displayed when the expression is evaluated. If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever". Write your answers within the designated space for each part.

(e) (2 pt) What will be displayed after the expression below is evaluated? Note that the `index()` method returns the index of the first occurrence of the given object. Each space within `motivation` should be counted as a single space.

```
motivation = "I got this! Python is easy! It takes time, but I'll get there!"
print(motivation.index('!'))
```

(f) (3 pt) `print(3 < 4 + 5 and 5 < 4)`

(g) (3 pt) `print((1 < 2) * 3 + (1 == 0.9999) + 1)`

(h) (3 pt) What will be displayed after the expression below is evaluated?

```
import numpy as np
array = np.array([0, 1, 3, 6, 7, 9])
print(array > 4)
```

Parts (i), (j), (k), and (l) are all related to the following function:

```
def force(mass, g=10, friction=0):
    f = mass * g + mass * g * friction
    return f
```

Assume the function above has been defined. For each part, write the output that will be displayed when the expression is evaluated. If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever". Write your answers within the designated space for each part.

(i) (2 pt) `print(force(1, 2, 1))`

(j) (2 pt) `print(force(1, friction=2))`

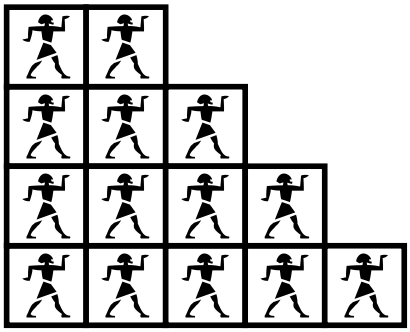
(k) (2 pt) `print(force(g=1, friction=2))`

(l) (2 pt) What will be displayed after the following code has been evaluated?

```
mass = 1
g = 2
friction = 0.5
friction = force(friction, mass, g)
print(friction)
```

Question 2: Software Engineer Egyptians Edition (20 points)

Write a function `pyramids()` which takes two positive integers: the number of available building blocks (`n`) and the number of blocks that should be used for the top layer (`t`). The function returns a tuple containing two values: (1) the number of layers in the pyramid (`layers`), and (2) the number of blocks that remain unused after constructing the pyramid. When constructing the pyramid, the top layer must contain `t` blocks, and each subsequent layer should have one more block than the layer directly above it. So, if you start with `n = 16` blocks and need to have `t = 2` blocks for the top layer, you can construct 4 layers ($2 + 3 + 4 + 5 = 14$ blocks) and will have 2 unused blocks.



Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

```
def pyramids(n, t):
    """Return the number of layers and unused blocks in a pyramid construction.

    >>> pyramids(1, 2) # not enough blocks to start the top layer
    (0, 1)

    >>> pyramids(10, 3) # 2 layers (3 + 4 = 7) + 3 unused blocks
    (2, 3)

    >>> pyramids(16, 2) # 4 layers (2 + 3 + 4 + 5 = 14) + 2 unused blocks
    (4, 2)
    """

    ______(a)______

    blocks_current_layer = ______(b)______ # blocks needed for the current layer

    while ______(c)______:

        ______(d)______

        ______(e)______

        blocks_current_layer += ______(f)______

    return layers, n
```

You will implement the function `pyramids()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. Except when explicitly provided as part of multiple-choice questions, you may not use any functions, built-in or otherwise. You may not modify the level of indentation of any line.

(a) (2 pt) Fill in blank (a).

(b) (2 pt) Fill in blank (b).

(c) (2 pt) Fill in blank (c).

(d) (2 pt) Which of these could fill in blank (d)?

- ◯ `layers + 1`
- ◯ `layers = 0`
- ◯ `layers == layers + 1`
- ◯ `layers = layers + 1`
- ◯ `layers = 2`

(e) (3 pt) Fill in blank (e).

(f) (2 pt) Fill in blank (f).

(g) (2 pt) Which lines can be swapped without changing the function's output? When two lines are swapped, assume that all other lines will stay the same. Also, only the positions of the mentioned lines will be switched and their contents should remain unmodified. Note that line numbers are indicated on page 6, to the left of the code. Select ALL that apply.

- ☐ We can swap lines 10 and 11 without changing the function's output
- ☐ We can swap lines 13 and 14 without changing the function's output
- ☐ We can swap lines 13 and 15 without changing the function's output
- ☐ We can swap lines 14 and 15 without changing the function's output
- ☐ None of the lines can be swapped without changing the function's output

(h) (1 pt) Your friend chose to disregard the instructions, which state that both `n` and `t` should be positive integers. They proceeded to test various input arguments to see if your code would break. Which of the following function calls will lead to an infinite loop?

- ◯ `pyramids(10 ** 10, 10)`
- ◯ `pyramids(10 ** 10, -10)`
- ◯ `pyramids(-10 ** 10, 10)`
- ◯ `pyramids(10 ** 10, 0.5)`
- ◯ None of the options lead to an infinite loop

(i) (2 pt) We need to modify `pyramids()` to additionally return the number of blocks that were used in the last layer that was built. For example, `pyramids(10, 3)` should now return `(2, 3, 4)`, where 2 is the number of built layers (3 + 4 = 7), 3 is the number of unused blocks, and 4 is the number of blocks used for the last built layer. Complete the return statement in the box below accordingly while leaving the rest of the function unchanged. For this part only, assume that at least one layer has been built.

```
return ____________________________________________________________
```

(j) (2 pt) We would like to set the default number of blocks for the top layer to be equal to 1. Complete the line below accordingly while leaving the rest of the function unchanged.

```
def pyramids_________________________:
```

Question 3: Twins, Triplets, or More? (10 points)

Each part below is independent from the other. For each part, select ALL that apply.

(a) (4 pt) Given the data `AQI = [138, 152, 187, 178, 151, 147]`, which of the following code snippets will display the same output when evaluated?

☐
```
for i in AQI:
    if i % 2 == 0:
        break
    print(i)
print(i)
```

☐
```
for j in AQI:
    if j >= 150:
        break
    print(j)
```

☐
```
for m in AQI:
    if not m > 145:
        print(m)
```

☐
```
for k in AQI:
    if k >= 150:
        continue
    print(k)
```

(b) (2 pt) Which of the following function definitions are 'synonyms', meaning, they can be used interchangeably because they yield the same results for the same inputs.

☐
```
def fun(n):
    return n + 2, n ** 2
```

☐
```
def fun(n):
    return n + 2 + n ** 2
```

☐
```
def fun(n):
    return n + 2
    return n ** 2
```

☐
```
def fun(n):
    return n ** 2, n + 2
```

☐
```
fun = lambda n : n + 2
```

☐ None of the options are synonyms

(c) (4 pt) Given the data `rain = (0.2, 0.3, 0, 1.2, 1.5)`, which of the following code snippets will display the same output when evaluated?

☐
```
for i in rain:
    print(i)
```

☐
```
for k in range(len(rain)):
    print(rain[k])
```

☐
```
for j in range(len(rain)):
    print(j)
```

☐
```
m = 0
while m < len(rain):
    print(rain[m])
    m += 1
```

Question 4: OK. We Need to Cool It Down (15 points)

You have been tasked with writing a program that controls the cooling of an electric pump. The code below has been developed to determine the time required for the temperature to reach a specific target. **For all parts, assume the following code has been executed.**

```
def convert(t, delta=0, factor=1):
    print(t) if t % 2 == 0 else None
    t = t - delta - delta * factor
    return t

temperature = 72
time = 0.5

while temperature > 68:
    t = temperature - 4.0
    convert(temperature - 2)
    temperature, time = temperature - 1, time * 2
    s = t
print("We're cool!")
```

The parts for this question begin on the next page. You can use the space below for scratch work. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

For all parts, assume the code on the previous page has been executed.

(a) (5 pt) What would Python display? The output may have multiple lines. If an error occurs, write "Error". If nothing is displayed, write "Nothing". If this runs forever, write "Forever".

(b) (2 pt) What is the final value associated with the variable `temperature` in the global frame?

(c) (2 pt) What is the final value associated with the variable `time` in the global frame?

(d) (2 pt) What is the final value associated with the variable `s` in the global frame?

(e) (3 pt) What is the final value associated with the variable `delta` in the global frame?

- ◯ `delta` is not defined in the global frame
- ◯ `1`
- ◯ `0`
- ◯ `-2`
- ◯ `temperature - 2`

(f) (1 pt) What is the result upon evaluating the expression: `type(s)`

- ◯ `int`
- ◯ `bool`
- ◯ `str`
- ◯ `None`
- ◯ `float`

Question 5: AQI Going Through the Roof (15 points)

After passing E7 with flying colors, you were recruited by the United States Environmental Protection Agency (EPA) to develop a program for monitoring the Air Quality Index (AQI). The AQI provides information about air pollution levels. A higher AQI value indicates greater air pollution and increased health concerns. An AQI greater than 150 is considered unhealthy for the general population. The EPA's objective is to determine the appropriate time to issue a warning based on high AQI values. However, since AQI values tend to fluctuate, EPA wants to issue a warning **only when AQI values have exceeded 150 for three consecutive measurements**.

Write a function `warning()` which takes a list of AQI values (`AQI`). The function should return the index at which AQI values exceed 150 for three consecutive measurements. Otherwise, if AQI values do not exceed 150 for any three consecutive measurements, the function should return 0. Additionally, if there are multiple sequences that meet this criterion, the function should consider only the first such sequence.

Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

```
def warning(AQI):
    """Return the index at which 3 consecutive measurements exceed 150.
    Return 0 otherwise.

    >>> warning([138, 152, 187, 155, 178, 151, 147])
    3

    >>> warning([138, 152, 187, 102, 178, 151, 147])
    0

    >>> warning([138, 152, 187, 102, 178, 151, 165])
    6
    """

    count = ______(a)______

    for i, value in ______(b)______:

        if value > 150:

            ______(c)______

        else:
            ______(d)______

        if ______(e)_____:

            return ______(f)______

    ______(g)______
```

You will implement the function `warning()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. Except when explicitly provided as part of multiple-choice questions, you may not use any functions, built-in or otherwise. You may not modify the level of indentation of any line.

(a) (2 pt) Fill in blank (a).

(b) (2 pt) Which of these could fill in blank (b)? Select ALL that apply.

- ☐ `AQI`
- ☐ `zip(AQI)`
- ☐ `enumerate(AQI)`
- ☐ `range(len(AQI))`
- ☐ `zip(AQI, range(len(AQI)))`
- ☐ `zip(range(len(AQI)), AQI)`

(c) (2 pt) Fill in blank (c).

(d) (2 pt) Fill in blank (d).

(e) (2 pt) Fill in blank (e).

(f) (3 pt) Fill in blank (f).

(g) (2 pt) Fill in blank (g).

Question 6: Recursion After Recursion (10 points)

For all parts, assume the following Python code has been executed.

```
def F(n):
    if n == 1:
        return 1
    else:
        return -n + F(n - 1)

def G(n):
    if n == 0:
        return 1
    elif n == 1:
        return 2
    else:
        return n - G(G(n - 2))

def H(n):
    if n == 0:
        return 1
    else:
        return n - H(n - 3)
```

The parts for this question begin on the next page. You can use the space below for scratch work. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

(a) (2 pt) Select the function(s) that return the same output as `F()` for `n > 0`.

☐
```
def Fk(n):
    result = 0
    for i in range(n, 0, -1):
        result = result + i
    return result
```

☐
```
def Fi(n):
    result = 0
    for i in range(n, 0, -1):
        result = result - i
    return result + 2
```

☐
```
def Fl(n):
    result = 0
    i = n
    while i >= 1:
        result -= n
        i -= 1
    return result
```

☐
```
def Fj(n):
    result = 0
    for i in range(n, 0, -1):
        if i == 1:
            result = result + i
        else:
            result = result - i
    return result
```

☐ None of the options

For parts (b), (c), (d), and (e), write the output that will be displayed when the given expression is evaluated. If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever". Write your answers within the designated spaces.

(b) (2 pt) `print(F(4))`

(c) (2 pt) `print([G(i) for i in range(3)])`

(d) (2 pt) `print(G(5))`

(e) (2 pt) `print(H(4))`